

AI NEXUS 360

Module 2 Assignment — Part A: Theory & Conceptual Understanding

Section A — Short Answer Questions

Q1. Generative AI Fundamentals

Generative AI refers to AI systems that create new content -- text, images, code, or audio -- rather than merely classifying or predicting on existing data. Traditional AI/ML systems are typically discriminative: given labeled data, they learn a mapping from inputs to a fixed set of outputs, such as spam/not-spam or a regression value. Generative AI, especially Large Language Models (LLMs), learns the underlying probability distribution of language (or another modality) and samples from it to produce novel, coherent output token by token. An LLM's role in a Generative AI application is that of the core reasoning and generation engine: it receives a prompt (instructions plus context), predicts the most probable continuation based on patterns learned from massive training data, and returns text the application then processes, validates, and presents to the user -- effectively acting as a flexible, general-purpose engine the app is built around.

Q2. Prompt Engineering

Prompt Engineering is the practice of designing and refining the input given to an LLM so its output is accurate, relevant, and usable, without changing the model itself. Because LLMs are highly sensitive to phrasing, structure, and context, a well-engineered prompt can be the difference between a vague, generic response and a precise, production-ready one. The major components of an effective prompt are: Role (who the model should act as), Context (background information needed to understand the task), Task (the specific instruction), Constraints (rules or boundaries the output must respect), Examples (optional demonstrations of the desired input/output), and Output Format (how the response should be structured, e.g. JSON, bullet list, word count). Combining these components deliberately, rather than writing one unstructured sentence, is what separates casual prompting from prompt engineering as an application-development discipline.

Q3. Zero-Shot, One-Shot and Few-Shot Prompting

Zero-Shot Prompting gives the model only an instruction and no examples, relying entirely on its pretrained knowledge to infer the desired format and style. Example: "Extract all function names from this Python file." One-Shot Prompting provides exactly one worked example before the actual task, anchoring the model to a specific output style. Example: "Convert to camelCase. Input: user_name -> Output: userName. Now convert: api_key." Few-Shot Prompting provides multiple (typically two to five) examples, helping the model infer more complex or nuanced patterns a single example can't fully capture. Example: showing three input/output pairs of buggy-code-to-fixed-code before asking it to fix a fourth snippet. In general, more examples reduce ambiguity but cost more tokens, so the choice is a trade-off between output reliability and context budget.

Q4. Role Prompting

Role Prompting instructs the LLM to adopt a specific persona or professional identity before performing a task, which shapes its vocabulary, priorities, and depth of reasoning even when the underlying task is unchanged. Asking "Explain recursion" with no role produces a generic, textbook-style answer. Asking the same question with "You are a strict CS professor grading a viva" produces a terser, more exacting explanation that probes for gaps. Asking it as "You are a friendly older sibling helping with homework" produces a warmer, analogy-heavy explanation instead. In AI NEXUS 360, the Resume Generator uses a "senior technical recruiter" role so output favors

ATS-friendly, achievement-oriented phrasing, while the Interview Assistant uses a "strict panel interviewer" role so evaluations are rigorous rather than generically encouraging -- the same underlying question produces meaningfully different responses purely from the assigned persona.

Q5. Structured Prompting

Structured Prompting organizes a prompt into explicit, labeled sections -- Role, Context, Task, Constraints, Output Format -- instead of a single free-form paragraph. This matters in production AI applications because unstructured prompts produce unpredictable output shapes, which is fine for a chatbot but breaks any code that needs to parse, store, or display the response programmatically. A structured prompt with an explicit "OUTPUT FORMAT: strict JSON with fields X, Y, Z" lets the Response Processing layer reliably parse the result with a JSON parser instead of writing fragile regexes to extract information from prose. It also makes prompts easier to test, version, and debug, since each section can be changed independently -- e.g. tightening Constraints without touching the Task description -- which is essential once a prompt is embedded inside an application rather than typed once into a chat window.

Q6. Chain-of-Thought and Tree-of-Thought

Chain-of-Thought (CoT) prompting asks the model to reason step by step through a problem before giving its final answer, improving accuracy on multi-step logical, mathematical, or diagnostic tasks by making intermediate reasoning explicit rather than jumping straight to a conclusion. Tree-of-Thought (ToT) extends this by having the model explore multiple independent reasoning branches in parallel -- for example, three different ways to solve a problem or phrase an answer -- evaluating each before selecting or recommending the best one. CoT is a single linear reasoning path, while ToT is a breadth-first exploration of several paths with explicit comparison. CoT suits problems with one correct reasoning route (debugging a race condition, solving a DSA problem), whereas ToT suits problems with several valid solutions where the best choice is subjective or context-dependent, such as phrasing a resume bullet three different ways and picking the strongest.

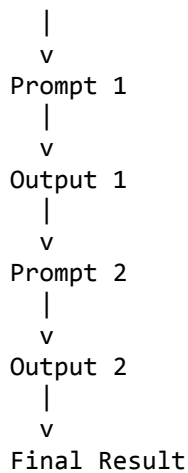
Q7. Meta Prompting

Meta Prompting is the technique of using an LLM to generate, refine, or critique another prompt, rather than to perform the end task directly. Instead of prompting the model to "summarize this article," a meta-prompt asks it to "design an effective prompt for summarizing news articles for a busy executive audience, including role, constraints, and output format." This is useful when the prompt engineer isn't sure what the ideal prompt structure is, or wants to systematically improve an existing weak prompt. A meta-prompt typically supplies the target task description, any known constraints, and an instruction to output a ready-to-use prompt rather than the task's own output. In AI NEXUS 360, a meta-prompt is used to have the model design new prompts for tasks like adaptive interview-question generation, where hand-designing every prompt state manually would be tedious.

Q8. Prompt Chaining

Prompt Chaining breaks a complex task into a sequence of smaller, focused prompts, where the output of one prompt becomes the input to the next, rather than asking a single large prompt to do everything at once. This differs from sending one large prompt in that each stage has a narrow, verifiable job -- easier to debug, more reliable, and reusable independently -- whereas a single mega-prompt asking for skill extraction, gap analysis, recommendations, and a rewritten resume all at once tends to produce shallower, less consistent results because the model must juggle too many instructions simultaneously.

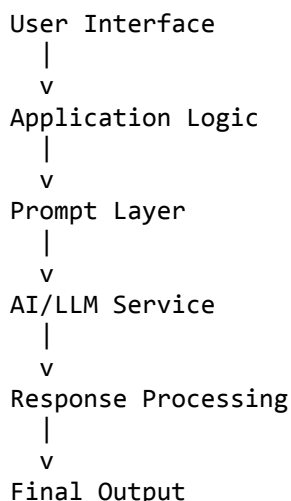
Input



AI NEXUS 360 implements this as the Resume Optimization Chain: Job Description leads into a prompt that extracts required skills, whose output feeds a prompt that analyzes the candidate's skill gap, whose output feeds a prompt that generates recommendations, whose output finally feeds a prompt that produces the optimized resume -- each stage built only from the prior stage's output, never re-sending the raw job description from scratch.

Q9. AI Application Architecture

An AI-powered application is typically structured in six layers. The User Interface (e.g. a Streamlit page) collects input and displays results, with no AI logic of its own. Application Logic (the services layer) validates input, decides which prompts to build and in what order, and coordinates multiple calls. The Prompt Layer is responsible purely for constructing well-formed prompt strings -- role, context, constraints, output format -- from the data it is given. The AI/LLM Service is the single point of contact with the model provider's API, handling authentication, retries, and errors. Response Processing takes the raw model output and parses it -- JSON decoding, Markdown section splitting, schema validation -- before it is usable. Finally, the Final Output is rendered back through the UI or exported as a file. Separating these layers means each can be tested, replaced, or reused independently -- swapping the LLM provider, for instance, only touches one file.



Q10. AI Application Development

Python is the glue language coordinating input validation, prompt construction, API calls, and output formatting across the application. Streamlit provides the web UI layer with minimal boilerplate -- widgets like text inputs and buttons map directly to Python variables, letting a backend-focused developer build a usable interface without separate frontend code. The API/LLM service (here, the

Anthropic API) is the actual generation engine, invoked through a thin wrapper that centralizes model selection and error handling. Environment variables (loaded via python-dotenv) keep secrets like API keys out of source code and version control, and allow different keys or models per environment without code changes. Structured output -- enforcing JSON or fixed Markdown sections -- is what makes LLM responses machine-consumable rather than just human-readable prose, enabling features like ATS scoring or numeric evaluation. Session state (st.session_state in Streamlit) persists data such as generated resumes or interview progress across reruns, since Streamlit re-executes the whole script on every interaction. Document generation (python-docx, plain-text writers) turns final structured content into downloadable deliverables like resumes and reports.

Section B — Conceptual Comparison

Q11. Prompting Technique Comparison

Technique	Purpose	Example Use Case
Zero-Shot	Get output from an instruction alone, no examples, for a well-understood task	Extracting ATS keywords from a job description
One-Shot	Anchor output style/format with a single worked example	Converting a resume bullet to a fixed phrasing style
Few-Shot	Teach a nuanced pattern via multiple examples one example can't capture	Scoring interview answers on a consistent rubric
Role Prompting	Shape tone, priorities and depth by assigning a persona	Recruiter persona critiquing a resume
Meta Prompting	Generate or improve another prompt rather than doing the task directly	Designing an adaptive interview-question prompt

Q12. Chain-of-Thought vs Tree-of-Thought

Aspect	Chain-of-Thought	Tree-of-Thought
Structure	Linear, single reasoning path	Branching, multiple parallel paths
Purpose	Improve accuracy on multi-step logic	Explore and compare multiple valid solutions
Best for	One correct route (debugging, math, DSA)	Several valid options (creative phrasing, design choices)
Output	One reasoning trail plus final answer	Several evaluated branches plus a recommendation
Cost	Lower token cost	Higher token cost (multiple branches evaluated)

Q13. Single Prompt vs Prompt Chaining

Aspect	Single Prompt	Prompt Chaining
--------	---------------	-----------------

Task complexity	Best for simple, single-step tasks	Best for multi-step, dependent tasks
Reliability	Can degrade as instructions pile up	Each stage stays focused, generally more reliable
Debuggability	Hard to isolate which instruction failed	Each stage's output can be inspected independently
Reusability	Instructions bundled together	Each stage/prompt is reusable elsewhere
Example	"Summarize, translate and format as JSON" in one call	Resume Optimization Chain (four separate stages)

Q14. Unstructured Output vs Structured Output

Aspect	Unstructured Output	Structured Output
Format	Free-form prose	Fixed schema (JSON, Markdown headings, tables)
Parseability	Hard to parse programmatically	Directly parseable, no regex needed
Use case	Chat responses, creative writing	Feeding UI components, databases, exports
Failure mode	Ambiguous phrasing, inconsistent formatting	Occasional schema violations if not enforced
Example here	Resume improvement suggestions text	Interview evaluation JSON scores

Q15. Keyword-based ATS Matching vs Semantic/AI-based Matching

Keyword-based ATS Matching checks for exact (or near-exact, substring/stemmed) matches between terms in a job description and terms in a resume -- e.g. checking whether the literal string "Python" appears. It is fast, transparent, and easy to explain, but brittle: a resume that says "Built ML models using PyTorch" would fail a check for the keyword "Machine Learning" even though it is clearly relevant. Semantic/AI-based Matching instead uses embeddings or an LLM to judge conceptual similarity, so it can recognize that "PyTorch experience" relates to "Machine Learning," or that "led a team of 4" implies leadership experience, even without exact wording. Semantic matching is more accurate for real-world resumes but less transparent and more computationally expensive. AI NEXUS 360 deliberately implements keyword-based matching for its ATS score, since it mirrors what most real-world commercial ATS systems actually use and keeps the scoring logic auditable, while listing semantic matching as a documented future enhancement.

Part B — Practical Implementation

The full practical implementation -- the AI Resume Generator, AI Interview Assistant, AI Content Generator, the 100-prompt library, the Resume Optimization Chain, and the README -- is delivered as source code (app.py, pages/, services/, prompts/, utils/, requirements.txt), prompt_library.md, and README.md alongside this document, per the submission structure required by the assignment. Running the application locally (streamlit run app.py) and capturing the screenshots listed in the

Practical Submission Requirements is the one step that has to happen on your machine, since it requires a live Anthropic API key and an interactive browser session.

- > (services,prompts,utils,pages)
- > pages
- > prompts
- > services
- > utils
- ⚙️ .env.example
- 📄 app.py
- 📄 build_prompt_library.py
- 📄 prompt_library.md
- 📖 README.md
- 📄 requirements.txt

```

Downloading jiter-0.17.0-cp312-cp312-win_amd64.whl (227 kB)
Using cached python_docx-1.2.0-py3-none-any.whl (252 kB)
Downloading lxml-6.1.3-cp312-cp312-win_amd64.whl (4.0 MB)
4.0/4.0 MB 208.5 kB/s 0:00:19
Downloading truststore-0.10.4-py3-none-any.whl (18 kB)
Installing collected packages: truststore, lxml, jiter, idna, docstring-parser,
httpx2, anthropic
  Attempting uninstall: idna
    Found existing installation: idna 3.11
    Uninstalling idna-3.11:
      Successfully uninstalled idna-3.11
Successfully installed anthropic-1.8.0 docstring-parser-0.18.0 httpcore2-2.13.1
jiter-0.17.0 lxml-6.1.3 python-docx-1.2.0 truststore-0.10.4

C:\Users\devan\Downloads\ai-nexus-360-project>streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://10.98.50.42:8501

```

The screenshot displays the AI NEXUS 360 application interface. On the left, a sidebar contains a navigation menu with the following items: "app", "Resume Generator", "Interview Assistant", "Content Generator", and "Prompt Chaining Demo". The main content area features a header with the "AI NEXUS 360" logo and title, followed by a subtitle "Generative AI, Prompt Engineering & AI Application Development". A welcome message states: "Welcome. This app is a mini GenAI ecosystem with three tools that all share one Prompt Library and one AI service layer:". Below this, a list of features is provided: "Resume Generator" (ATS-oriented resume generation + ATS scoring + improvement suggestions), "Interview Assistant" (role-based question generation, scored evaluation, performance report), "Content Generator" (multi-platform content generation from a single topic), and "Prompt Chaining Demo" (the Resume Optimization Chain, stage by stage). A note indicates that the sidebar is used for navigation and that each page collects its own inputs, with no shared state. The bottom section shows the "AI Resume Generator" form, which is divided into four steps: 1. Candidate Information (with fields for full name, email, location, and phone), 2. Professional Information (with fields for target role, technical skills, education, experience, projects, and certifications), 3. Job Description (with a field for the target job description), and 4. Generate Resume (with a "Generate Resume" button).

app

Resume Generator

Interview Assistant

Content Generator

Prompt Chaining Demo

AI Interview Assistant

1. Candidate Configuration

Target Role

Python Developer

Experience Level

Freelancer

Technology / Domain

Python + DSA

Interview Type

Technical

Start Interview

app

Resume Generator

Interview Assistant

Content Generator

Prompt Chaining Demo

AI Content Generator

Single Platform Multi Output (all platforms at once)

1. Content Request

Content Type

LinkedIn Post

Tone

Professional

Topic

Target Audience

Keywords (comma separated)

Content Length

Short

Generate Content

app

Resume Generator

Interview Assistant

Content Generator

Prompt Chaining Demo

Resume Optimization Chain

Job Description

Prompt 1: Extract Required Skills -> Output 1 (Required Skills)

Prompt 2: Analyze Candidate Skills -> Output 2 (Skill Gap)

Prompt 3: Generate Improvements -> Output 3 (Recommendations)

Prompt 4: Optimize Resume -> Final Resume

Job Description

Candidate's Current Skills

Draft Resume (paste the resume from the Resume Generator page)

Run the Chain